

Lezione 07: DDL & Intro a SQL

BASI DI DATI I

- Structured Query Language
- Data Definition Language – SQL
- Comandi DDL
- Qualche Esempio/Esercizio

A.A. 2022/2023

Prof. Silvio Barra

STRUCTURED QUERY LANGUAGE (SQL)



SQL

- Il linguaggio SQL permette la definizione, la manipolazione (aggiornamento e recupero) e la gestione di basi di dati relazionali.

- È una delle ragioni del successo dei db relazionali in ambito commerciale:

  essendo uno standard in tutti i DBMS relazionali, gli utenti sono poco propensi a migrare verso data model diversi, quali il gerarchico o il reticolare.



SQL – I VANTAGGI DI UNO STANDARD (1)

- SQL è considerato la principale ragione di successo dei database relazionali. Infatti:
  - **È uno standard** – è possibile migrare da un DBMS relazionale verso un altro DBMS relazionale senza eccessivi problemi.
  - I programmi applicativi usano lo stesso insieme di istruzioni SQL per accedere a DBMS relazionali diversi, senza dover cambiare i sottolinguaggi.
- *I costi di formazione del personale sono ridotti:* la formazione è concentrata su di un solo linguaggio.
- *Produttività:* i tecnici, una volta imparato il linguaggio (e poiché usano solo questo) diventano sempre più esperti e produttivi.
- *Portabilità delle applicazioni:* le applicazioni possono essere spostate da una macchina all'altra, se entrambe usano SQL.



SQL – I VANTAGGI DI UNO STANDARD (2)

- *Longevità delle applicazioni:* uno standard tende a rimanere in voga per diverso tempo e non c'è necessita di riscrivere le applicazioni.
- *Ridotta dipendenza da un singolo produttore:* quando non è usato un linguaggio proprietario, è più facile usare differenti produttori di DBMS.

  Di conseguenza il mercato sarà più competitivo ed i prezzi calano.
- *Comunicazione fra sistemi:* differenti DBMS e programmi applicativi possono comunicare più facilmente.



MA HA ANCHE DEGLI SVANTAGGI...

- Può in qualche modo limitare la creatività e l'innovazione.
- Può non incontrare tutte le necessità di un'industria.
- Può essere difficile da cambiare, poiché molti produttori hanno investito su di esso.
- L'utilizzo di speciali funzionalità aggiunte a SQL da qualche produttore può far perdere i vantaggi di cui alle slide precedenti.



STORIA DI SQL (1)

- **1970**: Codd propone il modello relazionale: iniziano esperimenti e ricerche per la realizzazione di linguaggi relazionali, cioè di linguaggi in grado di realizzare le caratteristiche del modello astratto.
  - Il primo risultato è ***SEQUEL*** *(**S**tructured **E**nglish **QUE**ry **L**anguag*e), definito **all'IBM Research** come linguaggio per interfacciarsi con una DB relazionale sperimentale (SYSTEM R)
    - Facile da imparare e utilizzare;
- **1976**: viene definita una versione rivista, il **SEQUEL/2,** ridenominata **SQL** (**S**tructured **Q**uery **L**anguage)
- **1979**: Il primo prodotto basato su SQL viene chiamato **Oracle**, lanciato dalla Relational Software, Inc.
- **1981:** IBM annuncia un prodotto SQL denominato **SQL/D**ata **S**ystem;
- **1983:** viene rilasciato il DBMS relazionale **DB2** compatibile con SQL/DS.



STORIA DI SQL (2)

- **1970**: Codd propone il modello relazionale: iniziano esperimenti e ricerche per la realizzazione di linguaggi relazionali, cioè di linguaggi in grado di realizzare le caratteristiche del modello astratto.
  - Il primo risultato è ***SEQUEL*** *(**S**tructured **E**nglish **QUE**ry **L**anguag*e), definito **all'IBM Research** come linguaggio per interfacciarsi con una DB relazionale sperimentale (SYSTEM R)
    - Facile da imparare e utilizzare;
- **1976**: viene definita una versione rivista, il **SEQUEL/2,** ridenominata **SQL** (**S**tructured **Q**uery **L**anguage)
- **1979**: Il primo prodotto basato su SQL viene chiamato **Oracle**, lanciato dalla Relational Software, Inc.
- **1981:** IBM annuncia un prodotto SQL denominato **SQL/D**ata **S**ystem;
- **1983:** viene rilasciato il DBMS relazionale **DB2** compatibile con SQL/DS.



STORIA DI SQL (2)

- **1986:** grazie all'ANSI (American National Standard Institute) e alla ISO (International Standard Organization) nasce **SQL-1** (anche detto SQL-86)
- **1992:** revisioni ed estensioni fanno nascere **SQL2** (anche detto SQL-92)
- **1999:** nasce **SQL3** meglio conosciuta come SQL
- **2003-2006:** due aggiornamenti successivi (SQL:2003 e SQL:2006) includono caratteristiche per l'XML
- **2008:** nasce SQL:2008 che introduce caratteristiche per le basi di dati ad oggetti, prequel dell'aggiornamento SQL:2011
- *Oggi SQL è implementato da tutti i principali fornitori di DBMS, ed è il linguaggio per database più usato al mondo*.
- Sfortunatamente ogni DBMS relazionale implementa un suo livello (o dialetto) di SQL, che è un'estensione o un sottoinsieme di un livello standard.



IL LINGUAGGIO SQL

- SQL fornisce *statement* per la definizione di dati, query e aggiornamenti, quindi è sia un DDL che un DML.

- Fornisce inoltre facility per definire viste e per ricavare indici.

- SQL può essere usato interattivamente (con maschere del DBMS) o essere incorporato (embedded) in programmi C, Cobol, Java, etc.



TERMINOLOGIA SQL

- SQL ha una terminologia diversa da quella classica relazionale:
  - Relazione → Tabella
  - Tupla → Riga
  - Attributo → Colonna



DEFINIZIONE DI DATI, SCHEMI E VINCOLI IN SQL



CONCETTI DI SCHEMA

- Il concetto di **schema** SQL è usato per raggruppare tabelle ed altri costrutti che appartengono alla stessa applicazione di database.

- Uno schema SQL è identificato da un **nome dello schema**, ed include un identificatore di autorizzazione per indicare l'utente proprietario dello schema, così come dei **descrittori** per ogni elemento dello schema.



CONCETTO DI SCHEMA (2)

- Uno schema include:
  - Tabelle
  - Domini
  - Viste
  - Altri costrutti, quali permessi di autorizzazione, etc.

- La sintassi per creare uno schema è

CREATE SCHEMA *nome_schema* **AUTHORIZATION** *nome_utente*

- Crea uno schema chiamato *nome_schema*, il cui proprietario è l'utente con account *nome_utente*.



IL COMANDO CREATE TABLE

- **CREATE TABLE** è usato per specificare una nuova relazione, assegnandole un **nome** ed **un insieme di attributi e vincoli.**

- Gli attributi sono specificati da un **nome**, un **tipo di dato** per definire il dominio dei valori, ed eventuali **vincoli**.

- In ultimo si specifica la chiave, i vincoli di integrità di entità e di integrità referenziale.

ISTANZA DI DATABASE RELAZIONALE

Un'istanza del database *"Company"*

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John		Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin		Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia		Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer		Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh		Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce		English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad		Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James		Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
		Houston
		Stafford
		Bellaire
		Sugarland

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

CREATE TABLE: *ESEMPIO*

```
CREATE TABLE EMPLOYEE
        ( FNAME          VARCHAR(15)      NOT NULL ,
          MINIT          CHAR ,
          LNAME          VARCHAR(15)      NOT NULL ,
          SSN            CHAR(9)          NOT NULL ,
          BDATE          DATE
          ADDRESS        VARCHAR(30) ,
          SEX            CHAR ,
          SALARY         DECIMAL(10,2) ,
          SUPERSSN       CHAR(9) ,
          DNO            INT              NOT NULL ,
    PRIMARY KEY (SSN) ,
    FOREIGN KEY (SUPERSSN) REFERENCES EMPLOYEE(SSN) ,
    FOREIGN KEY (DNO) REFERENCES DEPARTMENT(DNUMBER) ) ;
CREATE TABLE DEPARTMENT
        ( DNAME          VARCHAR(15)      NOT NULL ,
          DNUMBER        INT              NOT NULL ,
          MGRSSN         CHAR(9)          NOT NULL ,
          MGRSTARTDATE   DATE ,
        PRIMARY KEY (DNUMBER) ,
        UNIQUE (DNAME) ,
        FOREIGN KEY (MGRSSN) REFERENCES EMPLOYEE(SSN) ) ;
CREATE TABLE DEPT_LOCATIONS
        ( DNUMBER        INT              NOT NULL ,
          DLOCATION      VARCHAR(15)      NOT NULL ,
        PRIMARY KEY (DNUMBER, DLOCATION) ,
        FOREIGN KEY (DNUMBER) REFERENCES DEPARTMENT(DNUMBER) ) ;
```

```
CREATE TABLE PROJECT
        ( PNAME          VARCHAR(15)      NOT NULL ,
          PNUMBER        INT              NOT NULL ,
          PLOCATION      VARCHAR(15) ,
          DNUM           INT              NOT NULL ,
        PRIMARY KEY (PNUMBER) ,
        UNIQUE (PNAME) ,
        FOREIGN KEY (DNUM) REFERENCES DEPARTMENT(DNUMBER) ) ;
CREATE TABLE WORKS_ON
        ( ESSN           CHAR(9)          NOT NULL ,
          PNO            INT              NOT NULL ,
          HOURS          DECIMAL(3,1)     NOT NULL ,
        PRIMARY KEY (ESSN, PNO) ,
        FOREIGN KEY (ESSN) REFERENCES EMPLOYEE(SSN) ,
        FOREIGN KEY (PNO) REFERENCES PROJECT(PNUMBER) ) ;
CREATE TABLE DEPENDENT
        ( ESSN           CHAR(9)          NOT NULL ,
          DEPENDENT_NAME VARCHAR(15)      NOT NULL ,
          SEX            CHAR ,
          BDATE          DATE ,
          RELATIONSHIP   VARCHAR(8) ,
              PRIMARY KEY (ESSN, DEPENDENT_NAME) ,
              FOREIGN KEY (ESSN) REFERENCES EMPLOYEE(SSN) ) ;
```

Gli statement SQL2 per definire lo schema del database *"Company"*

DOMINI E TIPI DI DATI PER GLI ATTRIBUTI IN SQL



TIPI DI DATI E DOMINI

- Tipi di dati disponibili in SQL2:
  - Numerici
    - Interi (INTEGER o INT, SMALLINT)
    - Reali (FLOAT, REAL, DOUBLE PRECISION)
    - Numeri formattati (DECIMAL(i,j), DEC(i,j), NUMERIC(i,j))
      - i, detta precisione, indica il numero di cifre decimali.
      - j, detta scala, indica il numero di cifre dopo la virgola.
  - Stringhe di caratteri
    - A lunghezza fissa (CHAR(n), CHARACTER(n))
    - A lunghezza variabile (VARCHAR(n) o CHAR VARYING(n))
      - Per default n, il numero massimo di caratteri, è 1.



TIPI DI DATI E DOMINI (2)

- Stringhe di bit:
  - A lunghezza fissa (BIT(n))
  - A lunghezza variabile (BIT VARYING(n))
- DATE:
  - Ha dieci posizioni, con componenti YEAR, MONTH e DAY.
  - Formato *YYYY-MM-DD*.
- TIME:
  - Ha (almeno) otto posizioni con componenti HOUR, MINUTE e SECOND.
  - Formato *HH:MM:SS*.



DOMINI PERSONALIZZATI

- In SQL2 è possibile sia dichiarare il tipo di dato di un attributo, sia dichiarare il dominio.
  - Ciò semplifica il cambiamento di un tipo per un dominio usato più volte nello schema.

- ***Esempio:***
  - **CREATE DOMAIN** SSN_TYPE **AS CHAR**(9);
    e poi si usa SSN_TYPE per gli attributi
    - SSN e SUPERSSN di EMPLOYEE
    - MGRSSN e ESSN di DEPARTMENT
    - ESSN di WORKS_ON



SPECIFICA DEI VINCOLI DI BASE PER SQL



VINCOLI DI BASE

- I vincoli di base per SQL sono dei seguenti tipi:
  - Vincoli sui valori null e valori predefiniti
  - Vincoli sulle tuple (CHECK)
  - Vincoli sui domini degli attributi
  - Vincoli di chiave
  - Vincoli di integrità referenziale



VINCOLI SUI VALORI NULL E PREDEFINITI

- Poiché SQL consente che un attributo abbia valore **null**, se si vuole impedire ciò si usa il vincolo

  NOT NULL.

  - Tale vincolo è sempre specificato implicitamente per la chiave primaria *(vincolo di integrità di entità)*.

- È anche possibile specificare un valore di default per un attributo, attraverso la clausola

  DEFAULT <*value*>, dopo la dichiarazione dell'attributo

  - Senza tale clausola il valore di default di un attributo è null.



VINCOLI SULLE TUPLE (CHECK)

- Il vincolo CHECK serve a controllare che un determinato attributo rispetta determinate condizioni.
- La sintassi è CHECK(*vincolo*)

dove vincolo è una determinata condizione che deve essere rispettata

```
CREATE DOMAIN sesso AS CHAR
        CHECK(sesso ='m' OR sesso='f')
```

```
CREATE DOMAIN voto AS SMALLINT
        CHECK(voto> 0 AND voto<=30)
```

```
CONSTRAINT checkLode
        CHECK(lode=TRUE AND voro=30) OR NOT lode=TRUE
```



VINCOLI DI CHIAVE

- I vincoli di chiave sono importanti all'interno dell'istruzione **CREATE TABLE**
- Per specificare una chiave primaria, dopo la specifica dell'attributo è necessario inserire il vincolo **PRIMARY KEY.**

Es: CREATE TABLE IMPIEGATO

(SSN char(9) PRIMARY KEY, …

- Se più attributi compongono la chiave primaria, il vincolo si inserisce a valle della definizione della tabella

CONSTRAINT IMPIEGATO_PK PRIMARY KEY(nome, cognome)



VINCOLI DI UNICITÀ

- È anche possibile definire dei vincoli di chiave alternativa (vincolo di unicità) per specificare che un determinato attributo, o sequenza di attributo, deve avere valori unici all'interno della tabella.
- Il vincolo **UNIQUE** specifica una chiave secondaria

Es: CREATE TABLE DIPARTIMENTO

(Numero char(3) PRIMARY KEY,

Nome varchar(20) UNIQUE…

Oppure

CONSTRAINT IMPIEGATO_UNIQUE UNIQUE(Nome)



VINCOLI DI INTEGRITÀ REFERENZIALE

- Si specifica tramite la keyword **FOREIGN KEY** seguita dall'attributo che indica la chiave esterna, la parola **REFERENCES** e l'attributo esterno cui si riferisce

- **Es: CREATE TABLE impiegato(….**

  …

  …

  CONSTRAINT SUPERIMP_FK FOREIGN KEY(SUPERSSN) REFERENCES impiegato(SSN)



UTILIZZO DELLA KEYWORD CONSTRAINT

- L'utilizzo di **CONSTRAINT**, quando viene definito un vincolo è facoltativo
- Ma diventa particolarmente utile, in quanto premette di dare un nome ad un vincolo
  - Questo ci permette di manipolare il vincolo in maniera esplicita in seguito alla sua creazione



UTILIZZO DELLA KEYWORD CONSTRAINT

- L'utilizzo di **CONSTRAINT**, quando viene definito un vincolo è facoltativo
- Ma diventa particolarmente utile, in quanto premette di dare un nome ad un vincolo
  - Questo ci permette di manipolare il vincolo in maniera esplicita in seguito alla sua creazione

  - ***FORTEMENTE CONSIGLIATO!***



VINCOLI DI INTEGRITÀ REFERENZIALE

- Il progettista dello schema può specificare l'azione da intraprendere se si viola un vincolo di integrità referenziale, attraverso la cancellazione di una tupla referenziata o attraverso la modifica di un valore di chiave referenziata.

- **L'azione referenziale triggered** può essere specificata nella clausola **FOREIGN KEY**.
  - Possibili azioni sono **SET NULL**, **CASCADE** e **SET DEFAULT**, qualificate da opzioni **ON DELETE** e **ON UPDATE**.



ESEMPIO

Specifica di valori di default e azioni referenziali triggered.

```
CREATE TABLE EMPLOYEE
        ( . . . ,
          DNO              INT    NOT NULL    DEFAULT 1,
        CONSTRAINT EMPPK
          PRIMARY KEY (SSN) ,
        CONSTRAINT EMPSUPERFK
          FOREIGN KEY (SUPERSSN) REFERENCES EMPLOYEE(SSN)
                           ON DELETE SET NULL    ON UPDATE CASCADE ,
        CONSTRAINT EMPDEPTFK
          FOREIGN KEY (DNO) REFERENCES DEPARTMENT(DNUMBER)
                           ON DELETE SET DEFAULT    ON UPDATE CASCADE );

CREATE TABLE DEPARTMENT
        ( . . . ,
          MGRSSN    CHAR(9)  NOT NULL DEFAULT  '888665555' ,
          . . . ,
          CONSTRAINT DEPTPK
            PRIMARY KEY (DNUMBER) ,
          CONSTRAINT DEPTSK
            UNIQUE (DNAME),
          CONSTRAINT DEPTMGRFK
            FOREIGN KEY (MGRSSN) REFERENCES EMPLOYEE(SSN)
                  ON DELETE SET DEFAULT    ON UPDATE CASCADE );

CREATE TABLE DEPT_LOCATIONS
        ( . . . ,
          PRIMARY KEY (DNUMBER, DLOCATION),
          FOREIGN KEY (DNUMBER) REFERENCES DEPARTMENT(DNUMBER)
            ON DELETE CASCADE    ON UPDATE CASCADE ) ;
```



ESEMPIO (CONT.)

- Nell'esempio, per la chiave esterna SUPERSSN di EMPLOYEE ci sono i vincoli:
  - **SET NULL ON DELETE**
    - Se la tupla dell'impiegato che supervisiona viene cancellata, il valore di SUPERSSN è posto a null in tutte le tuple impiegato che lo referenziano.
  - **CASCADE ON UPDATE**
    - Se il valore SSN di un impiegato che supervisiona è aggiornato, il nuovo valore è riportato in SUPERSSN di tutte le tuple impiegato che referenziano il valore aggiornato.
- Ai vincoli può essere dato un nome (per poterli riutilizzare), usando la keyword **COSTRAINT**.



RECAP VINCOLI

- CONSTRAINT <nome_vincolo> <vincolo>
- <vincolo>: = CHECK <espressione booleana>

  UNIQUE (<lista_attributi>)

  PRIMARY KEY (<lista attributi>)

  FOREIGN KEY (<lista_attributi_FK>) REFERENCES <nome_tabella>(lista_att_PK)

  [ON DELETE <azione>]

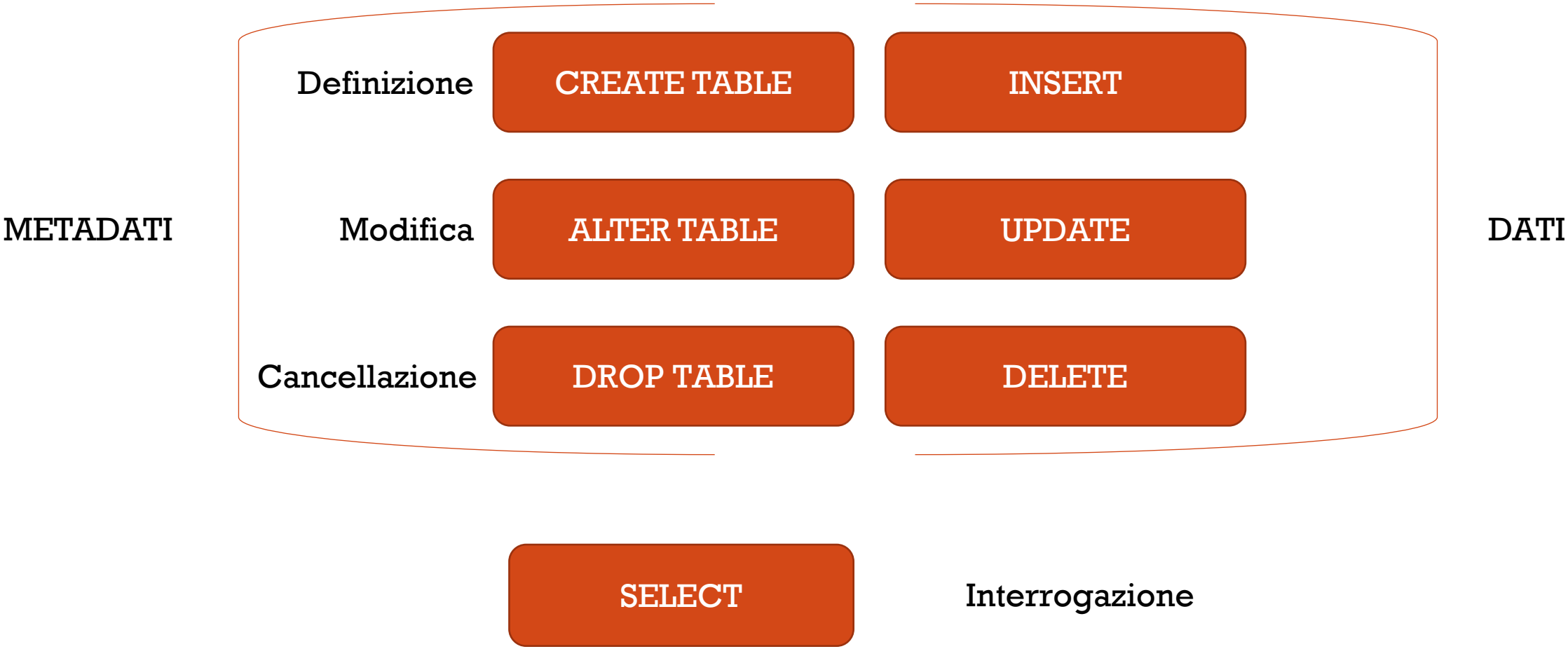
  [ON UPDATE <azione>]
- <azione>:= NO ACTION | CASCADE | SET DEFAULT | SET NULL



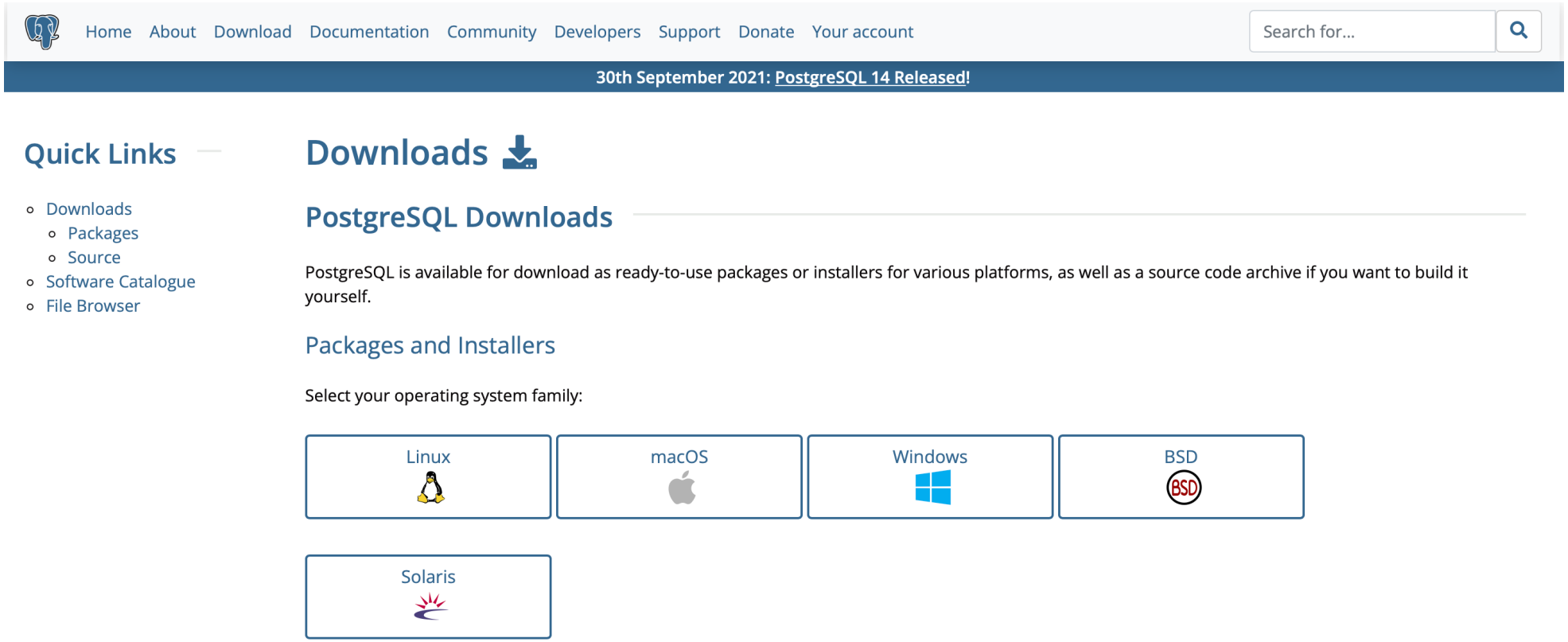
```
CREATE Table impiegato(
CF codice_fiscale,
fname VARCHAR(15),
mname CHAR,
lname VARCHAR(15),
gender CHAR,
dno int,
salary DECIMAL(10,2),
CONSTRAINT emppk PRIMARY KEY(CF),
CONSTRAINT fnameNN CHECK(fname IS NOT NULL),
CONSTRAINT lnameNN CHECK(lname IS NOT NULL),
CONSTRAINT depFK FOREIGN KEY(dno) REFERENCES dipartimento(dnumber),
CONSTRAINT genderCorrect CHECK(gender='m' OR gender='f'),
CONSTRAINT salaryCheck CHECK(salary>1000.00),
CONSTRAINT cfnumberUNIQUE UNIQUE(CF, dno)
)
```



DA DDL A SQL


METADATI
DATI
Definizione
Modifica
Cancellazione
CREATE TABLE
INSERT
ALTER TABLE
UPDATE
DROP TABLE
DELETE
SELECT
Interrogazione

POSTGRGES


Home About Download Documentation Community Developers Support Donate Your account
Search for...
30th September 2021: PostgreSQL 14 Released!
Quick Links
Downloads
Packages
Source
Software Catalogue
File Browser
Downloads
PostgreSQL Downloads
PostgreSQL is available for download as ready-to-use packages or installers for various platforms, as well as a source code archive if you want to build it yourself.
Packages and Installers
Select your operating system family:
Linux
macOS
Windows
BSD
Solaris

ESEMPIO 1

- Definire un dominio che permetta di rappresentare stringhe di lunghezza massima pari a 256 caratteri, su cui non sono ammessi valori nulli e con valore di default "*sconosciuto*".

SOLUZIONE 1

- CREATE DOMAIN String AS CHARACTER VARYING (256) DEFAULT ‘sconosciuto’ NOT NULL;



ESEMPIO 2

- Dare le definizioni SQL delle tre entità:

  FONDISTA(Nome, Nazione, Età*)

  GAREGGIA(NomeFondista↑, NomeGara ↑, Piazzamento)

  GARA(Nome, Luogo, Nazione, Lunghezza*)

rappresentando in particolare i vincoli di foreign key della tabella GAREGGIA.

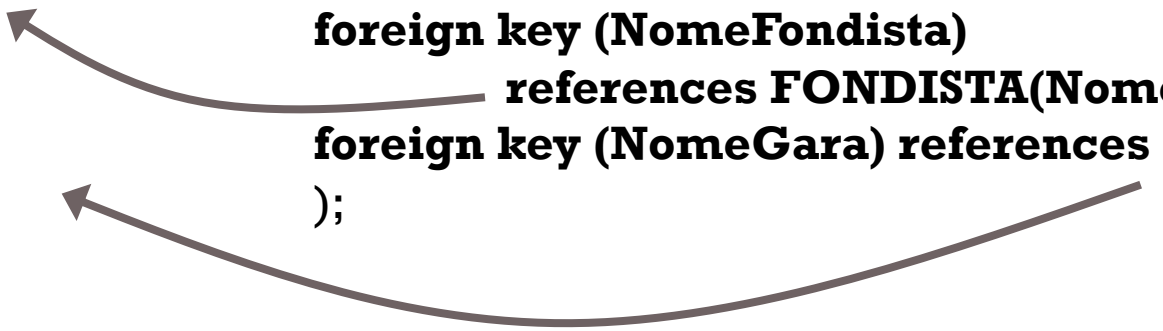


SOLUZIONE 2

```
CREATE TABLE FONDISTA
(
Nome varchar(20) not null,
Nazione varchar(30) not null,
Età smallint,
primary key (Nome)
);

CREATE TABLE GARA
(
Nome varchar(20) not null,
Luogo varchar(20) not null,
Nazione varchar(30) not null,
Lunghezza integer,
primary key (Nome)
);
```

```
CREATE TABLE GAREGGIA
(
NomeFondista varchar(20) not null,
NomeGara varchar(20) not null,
Piazzamento smallint,
primary key (NomeFondista, NomeGara),
foreign key (NomeFondista)
        references FONDISTA(Nome)
foreign key (NomeGara) references GARA(Nome)
);
```



ESEMPIO 3

- Dare le definizioni SQL delle entità:

  AUTORE (Nome, Cognome, DataNascita*, Nazionalità*)

  LIBRO (TitoloLibro, NomeAutore↑, CognomeAutore↑, Lingua*)

- Per il vincolo foreign key specificare una politica di cascade sulla cancellazione e di set null sulle modifiche.



SOLUZIONE 3

```
CREATE TABLE AUTORE
(
Nome varchar(20) not null,
Cognome varchar(20) not null,
DataNascita date,
Nazionalità varchar(30),
primary key(Nome, Cognome)
);
```

```
CREATE TABLE LIBRO
(
TitoloLibro varchar(30) not null,
NomeAutore varchar(20) ,
CognomeAutore varchar(20),
Lingua varchar(20),
primary key (TitoloLibro),
foreign key (NomeAutore, CognomeAutore)
   references AUTORE(Nome, Cognome)
   on delete cascade
   on update set NULL
);
```



INFO SULLA LEZIONE

- Capitolo 7
  - Fino al 7.2.4



COME MANIPOLARE UN DB RELAZIONALE

- L'algebra relazionale è il mezzo per interrogare basi di dati relazionali, ma non è sfruttabile in ambito commerciale, principalmente perché le query sono scritte specificando l'ordine con cui devono essere eseguite le operazioni.

- SQL è un linguaggio dichiarativo, basato in parte sull'algebra relazionale ed in parte sul calcolo relazionale.
  - L'utente specifica quale risultato deve essere raggiunto.
  - Il DBMS decide l'ordine delle operazioni.



RELAZIONI BASE E VIRTUALI

- Le relazioni create con CREATE TABLE sono dette tabelle base o relazioni base in SQL, e significa che sono create e memorizzate come file dal DBMS.

- Le relazioni base sono distinte dalle relazioni virtuali, create mediante **CREATE VIEW**, cui può o meno corrispondere un file fisico.

- In SQL gli attributi sono considerati ordinati nella sequenza in cui sono stati specificati.
  Le righe non sono considerate ordinate.



IL COMANDO DROP SCHEMA

- Se uno schema non è più necessario, si usa il comando **DROP SCHEMA**, con due possibili opzioni (*drop behaviour*): **CASCADE** e **RESTRICT**.

- ***Esempi:***
  - **DROP SCHEMA** COMPANY **CASCADE**;
    - Lo schema del db COMPANY viene rimosso, con tutte le tabelle, domini ed altri elementi.
  - **DROP SCHEMA** COMPANY **RESTRICT**;
    - Lo schema è eliminato solo se non contiene elementi.



IL COMANDO DROP TABLE

- Permette di eliminare una tabella:
  - **DROP TABLE** DEPENDENT **CASCADE**;
    - Se non si vuole tenere più traccia delle persone a carico nel db COMPANY.
  - **DROP TABLE** DEPENDENT **RESTRICT**;
    - La tabella è eliminata solo se non è referenziata in alcun vincolo o vista.
    - Con l'opzione CASCADE sarebbero automaticamente eliminati insieme alla tabella stessa.



IL COMANDO ALTER TABLE

- La definizione di una tabella base può essere cambiata usando il comando **ALTER TABLE**.

- Possibili azioni di modifica di una tabella sono:
  - Aggiunta o rimozione di attributi;
  - Cambio di definizione di una colonna;
  - Aggiunta o rimozione di un vincolo.



ALTER TABLE: *ESEMPIO*

- Vogliamo aggiungere il lavoro svolto da un impiegato nella tabella Employee:
  - **ALTER TABLE** COMPANY.EMPLOYEE **ADD** JOB VARCHAR(12);
- Il valore di JOB o si specifica di **default** o sarà **null**.
- *Con la ALTER TABLE non è permessa la clausola NOT NULL.*



ALTER TABLE: *ESEMPIO* (2)

- Vogliamo eliminare una colonna: occorre scegliere l'opzione CASCADE o RESTRICT:
  - Con CASCADE tutti i vincoli e le viste che referenziano la colonna sono eliminati automaticamente dallo schema.
  - Con RESTRICT il comando ha successo solo se nessun vincolo o vista referenzia la colonna.

- ***Esempio:*** rimuovere ADDRESS dalla tabella EMPLOYEE:
  - **ALTER TABLE** COMPANY.EMPLOYEE **DROP** ADDRESS CASCADE;



ALTER TABLE: *ESEMPIO* (3)

- Modifica di una colonna eliminando una clausola di default o definendone una nuova.
- ***Esempio:***
  - **ALTER TABLE** COMPANY.DEPARTMENT **ALTER** MGRSSN **DROP DEFAULT**;
  - **ALTER TABLE** COMPANY.DEPARTMENT **ALTER** MGRSSN **SET DEFAULT** "333444555";
- Cambio di vincoli:
  - È possibile eliminare un vincolo solo se ha un nome:
    - **ALTER TABLE** COMPANY.EMPLOYEE **DROP CONSTRAINT** EMPSUPERFK **CASCADE**;



ESEMPIO 4

- Dato lo schema dell'esercizio precedente, spiegare cosa può capitare con l'esecuzione dei seguenti comandi di aggiornamento:
  1. DELETE FROM AUTORE WHERE Cognome = 'Rossi';
  2. UPDATE LIBRO SET NomeAutore= 'Umberto' WHERE CognomeAutore = 'Eco';
  3. INSERT INTO AUTORE(Nome,Cognome) VALUES('Ugo', 'Bianchi');
  4. UPDATE AUTORE SET Nome = 'Italo' WHERE Cognome = 'Calvino';



SOLUZIONE 4

1. Il comando cancella dalla tabella AUTORE tutte le tuple con Cognome = 'Rossi'. A causa della politica cascade anche le tuple di LIBRO con CognomeAutore = 'Rossi' verranno eliminate.
2. Il comando modifica la tabella LIBRO per tutti i cognomi di autori 'Eco' viene settato il nome.
3. Il comando aggiunge una nuova tupla alla tabella AUTORE. *Non ha alcun effetto sulla tabella LIBRO.*
4. Le tuple di AUTORE con Cognome = 'Calvino' vengono aggiornate a Nome = 'Italo'. A causa della politica set null gli attributi NomeAutore e CognomeAutore delle tuple di LIBRO con CognomeAutore = 'Calvino' vengono posti a NULL.



ESEMPIO 5

- Date le definizioni:

CREATE DOMAIN Dominio AS INTEGER DEFAULT 10

CREATE TABLE Tabella (Attributo Dominio DEFAULT 5);

- indicare cosa avviene in seguito ai comandi:
  1. ALTER TABLE Tabella ALTER COLUMN Attributo DROP DEFAULT;
  2. ALTER DOMAIN Dominio DROP DEFAULT;
  3. DROP DOMAIN Dominio;



SOLUZIONE 5

1. Il comando cancella il valore di default di Attributo. Il valore di default dopo il comando sarà quello impostato in Dominio, ossia 10.

2. Il comando cancella il valore di default di Dominio. Dopo l'operazione il valore di default sarà NULL.

3. Il comando cancella l'intero dominio Domain. In Tabella il dominio di Attributo diventerà INTEGER.



ESERCIZIO

- Dare le definizioni SQL delle relazioni

Rappresentanti(CodRappr, Cognome, Nome, Via, Citta, Provincia, TotProvvigioni,PercentProvv)

Clienti (CodCliente, Cognome, Nome, Via, Citta, Provincia, Saldo, Fido, CodRapp ↑)

Ordini (NroOrdine, Data, CodCliente ↑)

DettOrdini (NroOrdine↑, NroArt↑, Qta, Prezzo)

Articoli(NroArt, Descrizione, Giacenza, Categoria, PrezzoUnitario)

dimensionando opportunamente gli attributi ed indicando i vincoli sia intrarelazionali che interrelazionali



INFO SULLA LEZIONE

- Capitolo 8
  - Solo l' 8.4



FINE

Per eventuali domande: (in ordine di preferenza personale)

- Ora.
- Chat di Teams
- Mail: silvio.barra@unina.it